Basics Day 4 — Session 4 (Hours 13–16)

Python Programming (Basic) • Lists, Booleans, Conditionals & Checkpoint 1

Session 4 Overview

Topics Covered Today

- Hour 13: Lists fundamentals — create, index, slice, and mutate
- Hour 14: Boolean expressions, comparison operators, and logical operators
- Hour 15: If / elif / else flow and nested conditions
- Hour 16: Checkpoint 1 — Receipt generator integration assessment

Hour 13: Lists Fundamentals

Learning Outcomes

- Explain what a list is in plain, beginner-friendly language
- Create a list literal and print it
- Access items with indexing (including negative indices)
- Retrieve a portion of a list with slicing
- Mutate a list with `append()`, `remove()`, and `pop()`
- Check whether an item is present using the `in` operator

The Problem Lists Solve

One value at a time — the old way

```
item_1 = "milk"
item_2 = "bread"
item_3 = "eggs"
item_4 = "tea"
item_5 = "rice"
```

The list way

```
shopping_list = ["milk", "bread", "eggs", "tea", "rice"]
print(shopping_list)
```

> A list is **one variable** that holds **multiple related values in order**.

Creating Lists

List literal syntax

```
fruits = ["apple", "banana", "cherry"]
print(fruits)
# ['apple', 'banana', 'cherry']
```

Empty list (start empty, grow later)

```
shopping_list = []
print(shopping_list)   # []
```

A list of numbers

```
numbers = [10, 20, 30]
print(numbers)
# [10, 20, 30]
```

Indexing Lists

Zero-based positions

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])   # apple
print(fruits[1])   # banana
print(fruits[2])   # cherry
```

Negative indexing (count from the end)

```
print(fruits[-1])   # cherry  (last item)
print(fruits[-2])   # banana  (second-to-last)
```

⚠️ Accessing an out-of-range index raises `IndexError` — stay within bounds!

Slicing Lists

Pattern: `list[start:end]` — end is not included

```
letters = ["a", "b", "c", "d", "e"]
print(letters[0:3])   # ['a', 'b', 'c']
print(letters[1:4])   # ['b', 'c', 'd']
print(letters[:2])    # ['a', 'b']
print(letters[2:])    # ['c', 'd', 'e']
```

Indexing vs. slicing

```
print(letters[2])     # 'c'        — one item
print(letters[2:4])   # ['c','d'] — a new list
```

> Indexing gives **one item**; slicing gives **a new list**.

Lists Are Mutable

Update an item by index

```
fruits = ["apple", "banana", "cherry"]
print(fruits)              # ['apple', 'banana', 'cherry']

fruits[1] = "blueberry"
print(fruits)              # ['apple', 'blueberry', 'cherry']
```

Strings are NOT mutable this way

```
word = "cat"
# word[0] = "b"   ← TypeError! Strings don't allow this
```

> **Mutable** = the list can be changed after it is created.
> Lists allow item-by-item updates; strings do not.

Core List Methods

`append()` — add to the end

```
shopping_list = ["milk", "bread"]
shopping_list.append("eggs")
print(shopping_list)   # ['milk', 'bread', 'eggs']
```

`remove()` — remove by value

```
shopping_list.remove("bread")
print(shopping_list)   # ['milk', 'eggs']
```

`pop()` — remove by index (default: last item)

```
shopping_list = ["milk", "bread", "eggs"]
removed = shopping_list.pop(1)   # removes index 1
print(removed)                   # bread
print(shopping_list)             # ['milk', 'eggs']
```

The `in` Operator and Safe Removal

Membership test

```
shopping_list = ["milk", "bread", "eggs"]
print("milk" in shopping_list)   # True
print("tea" in shopping_list)    # False
```

Safe removal pattern (avoid crash if item is missing)

```
item_to_remove = "tea"

if item_to_remove in shopping_list:
    shopping_list.remove(item_to_remove)
    print("Item removed.")
else:
    print("That item is not in the list.")
```

⚠️ Calling `remove()` on a missing item raises a `ValueError` — always check first!

Demo: Build a Shopping List Step by Step

```
shopping_list = []
print("Starting list:", shopping_list)

shopping_list.append("milk")
shopping_list.append("bread")
shopping_list.append("eggs")
print("After adds:", shopping_list)

shopping_list[1] = "whole grain bread"
print("After update:", shopping_list)

item_to_remove = "milk"
if item_to_remove in shopping_list:
    shopping_list.remove(item_to_remove)
print("After remove:", shopping_list)

popped_item = shopping_list.pop()
print("Popped:", popped_item)
print("Final list:", shopping_list)
print(f"Item count: {len(shopping_list)}")
```

Lab: Shopping List Program

Time: 25–35 minutes

Tasks

1. Start with an empty list
2. Ask the user for **5 shopping items** one at a time; `append()` each
3. Ask for **one item to remove**; use the safe membership-check pattern
4. Print the **final list** and **item count** using `len()`

Completion Criteria

✓ Program starts with an empty list and grows it with user input
✓ Uses `in` to check before calling `remove()`
✓ Prints a clean final list and count
✓ No crashes on any valid input sequence

Hour 14: Boolean Expressions & Comparison Operators

Learning Outcomes

- Use all six comparison operators to compare values
- Distinguish between assignment (`=`) and equality (`==`)
- Combine conditions with `and` , `or` , and `not`
- Predict the result of compound boolean expressions
- Avoid common off-by-one and type-mismatch pitfalls

The Six Comparison Operators

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 3`	`True`
`<`	Less than	`3 < 5`	`True`
`>`	Greater than	`5 > 3`	`True`
`<=`	Less than or equal	`5 <= 5`	`True`
`>=`	Greater than or equal	`5 >= 3`	`True`

Every comparison returns `True` or `False` — a **boolean** value.

`=` vs `==` — The Most Common Beginner Mistake

```
age = 18          # Assignment: puts 18 INTO age
age == 18         # Comparison: ASKS "is age equal to 18?"
```

What goes wrong

```
# ❌ SyntaxError — can't assign inside a condition
if age = 18:
    print("Adult")

# ✓ Correct
if age == 18:
    print("Adult")
```

> **Memory trick**: Single `=` puts a value **in**.
> Double `==` asks a **question**.

Logical Operators: `and`, `or`, `not`

`and` — both must be `True`

```
age >= 18 and has_license       # True only if BOTH are True
```

`or` — at least one must be `True`

```
is_weekend or is_holiday        # True if EITHER is True
```

`not` — flips the boolean

```
is_adult = True
not is_adult                    # False
```

Combining all three

```
# Adult (18+) AND (has membership OR paid fee)
```

Truth Tables at a Glance

`and` — only TT → True

A	B	A and B
T	T	**T**
T	F	F
F	T	F
F	F	F

`or` — only FF → False

A	B	A or B
T	T	T
T	F	T

Demo: Age-Gating Eligibility Checker

```
# Basic comparison
age = 16
print(f"Age >= 18? {age >= 18}")          # False

age = 25
print(f"Age >= 18? {age >= 18}")          # True

# Combining with and
has_license = True
print(f"Eligible? {age >= 18 and has_license}")  # True

has_license = False
print(f"Eligible? {age >= 18 and has_license}")  # False

# Full eligibility: adult AND (membership OR paid fee)
age = 20
has_membership = False
paid_fee = True
eligible = age >= 18 and (has_membership or paid_fee)
print(f"Eligible: {eligible}")            # True
```

Common Pitfalls (Hour 14)

Off-by-one boundary error

```
age = 18
if age > 18:      # ❌ Excludes exactly 18
    print("Adult")

if age >= 18:     # ✓ Includes 18
    print("Adult")
```

Type mismatch

```
age = "18"
if age > 18:            # ❌ TypeError — str vs int
    print("Adult")

age = int(input("Age: "))
if age >= 18:           # ✓ Both integers
    print("Adult")
```

Lab: Camp Eligibility Checker

Time: 20 minutes

Rules

- **Eligible** if: age between 8–16 (inclusive) AND swimming test passed AND parent contact on file

Tasks

1. Ask for age, swimming test result, parent contact (use `True` / `False`)
2. Build the eligibility expression
3. Print `"Eligible for camp!"` or `"Not eligible."`

Sample solution skeleton

```
age = int(input("Age: "))
```

Hour 15: If / Elif / Else Flow and Nested Conditions

Learning Outcomes

- Write `if`, `elif`, and `else` to create branching logic
- Understand that only **one** block executes per conditional chain
- Order conditions from most specific to most general
- Identify and fix unreachable code branches
- Use nested conditionals appropriately (and know when to use `and` instead)

The If / Elif / Else Structure

```
if condition_A:
    # runs ONLY if A is True
elif condition_B:
    # runs ONLY if A is False AND B is True
elif condition_C:
    # runs ONLY if A and B are False AND C is True
else:
    # runs if NO condition above was True
```

Critical rule: Python checks top-to-bottom. The **first** `True` branch runs; all others are skipped.

Example

```
age = 16
if age < 13:
    print("You are a child.")
elif age < 18:
```

Tiered Decisions: Shipping Cost

```
weight = 25

if weight <= 10:
    cost = 0        # Free
elif weight <= 50:
    cost = 5        # Standard
elif weight <= 100:
    cost = 12       # Heavy
else:
    print("Too heavy. Contact support.")
    cost = None

print(f"Shipping cost: ${cost}")
# Output: Shipping cost: $5
```

Trace through:

1. `weight <= 10`? No (25 is not ≤ 10)

Order Matters — Avoid Unreachable Code

❌ Unreachable second branch

```
x = 15
if x > 0:
    print("x is positive")
elif x > 10:            # ← never reached! x > 10 implies x > 0
    print("x > 10")
```

✓ Fix: most specific first

```
x = 15
if x > 10:
    print("x is greater than 10")
elif x > 0:
    print("x is positive (but ≤ 10)")
else:
    print("x is not positive")
```

Nested Conditionals

When to nest — second check depends on the first

```
age = 25
has_license = True

if age >= 18:
    if has_license:
        print("You can drive.")
    else:
        print("You're an adult but can't drive yet.")
else:
    print("You must be an adult to drive.")
# Output: You can drive.
```

Prefer flat `and` when both conditions are equal weight

```
# Nested (harder to read):
if age >= 18:
```

Demo: Shipping Cost Calculator with Boundaries

```
weight = float(input("Package weight (lbs): "))

if weight <= 10:
    cost = 0
elif weight <= 50:
    cost = 5
elif weight <= 100:
    cost = 12
else:
    print(f"Weight: {weight} lbs — Too heavy. Contact support.")
    cost = None

if cost is not None:
    print(f"Weight: {weight} lbs, Shipping cost: ${cost}")
```

Boundary test values to run:

- 9 lbs → $0 · 10 lbs → $0 · 11 lbs → $5 · 50 lbs → $5 · 51 lbs → $12 · 101 lbs → error

Guided Practice: Grade Assignment

```python
score = 87

if score >= 90:
    grade = "A"
    message = "Excellent!"
elif score >= 80:
    grade = "B"
    message = "Good job!"
elif score >= 70:
    grade = "C"
    message = "Satisfactory."
elif score >= 60:
    grade = "D"
    message = "Needs improvement."
else:
    grade = "F"
    message = "See me after class."

print(f"Score: {score}")
print(f"Grade: {grade}")
print(f"Message: {message}")
```

Lab: Shipping Calculator

Time: 18 minutes

Tiers

Weight	Cost
0–5 lbs	$3
5–15 lbs	$7
15–30 lbs	$12
30–50 lbs	$18
Over 50 lbs	Cannot ship

Tasks

1. Ask user for weight (`float`)

Hour 16: Checkpoint 1 — Receipt Generator

Learning Outcomes

- Integrate input handling, type conversion, arithmetic, and formatted output
- Demonstrate mastery of Checkpoint 1 by building a receipt generator
- Apply correct order of operations for multi-step calculations
- Test programs systematically using multiple input cases
- Reflect on strengths and areas for continued growth

What Checkpoint 1 Measures

Checkpoint 1 IS about:

- Taking requirements and turning them into code
- Using correct operators and data types
- Producing readable, formatted output
- Catching and fixing your own mistakes

Checkpoint 1 is NOT about:

- Memorizing exact syntax
- Being perfect on the first run
- Using advanced features outside Basics scope

> Open-book. You can reference your notes.
> The code must be yours.

The Receipt Generator Problem

Input: item name, quantity, price per item

Calculations:

- `subtotal = quantity × price_per_item`
- `subtotal_with_fee = subtotal + 2.50`
- `tax = subtotal_with_fee × 0.08`
- `total = subtotal_with_fee + tax`

Expected output (Widget, qty 3, $10.00):

```
========== RECEIPT ==========
Item: Widget
Quantity: 3
Price per item: $10.00
Subtotal: $30.00
Service fee: $2.50
```

Step-by-Step: Input and Type Conversion

```python
# Step 1: Get input
item = input("Item name: ")
quantity_str = input("Quantity: ")
price_str = input("Price per item: ")

# Step 2: Convert to numbers
quantity = float(quantity_str)
price_per_item = float(price_str)

# Step 3: Verify
print(f"Item: {item}")
print(f"Quantity: {quantity}")
print(f"Price: ${price_per_item}")
```

> `input()` always returns a **string**.
> You must convert to `float` before doing any math.

Step-by-Step: Calculations

```
# Subtotal
subtotal = quantity * price_per_item

# Add fixed service fee
subtotal_with_fee = subtotal + 2.50

# Calculate 8% tax on the full subtotal-with-fee
tax = subtotal_with_fee * 0.08

# Final total
total = subtotal_with_fee + tax
```

Verify with: Widget, 3, $10.00

Step	Value
Subtotal	$30.00
With fee	$32.50

Step-by-Step: Formatted Output

```
print("========== RECEIPT ==========")
print(f"Item: {item}")
print(f"Quantity: {quantity}")
print(f"Price per item: ${price_per_item:.2f}")
print(f"Subtotal: ${subtotal:.2f}")
print(f"Service fee: $2.50")
print(f"Subtotal with fee: ${subtotal_with_fee:.2f}")
print(f"Sales tax (8%): ${tax:.2f}")
print(f"Total: ${total:.2f}")
print("============================")
```

> `:.2f` inside an f-string formats a float to **exactly 2 decimal places**.
> Calculate first; round only when printing.

Checkpoint 1 Pitfalls

Forgetting to convert input

```
# ❌ TypeError — can't multiply a string by a number
quantity = input("Quantity: ")
subtotal = quantity * price_per_item

# ✓ Fix
quantity = float(input("Quantity: "))
```

Taxing the wrong amount

```
# ❌ Forgets to tax the service fee
tax = subtotal * 0.08

# ✓ Tax on the full subtotal-with-fee
tax = subtotal_with_fee * 0.08
```

Rounding too early

Checkpoint 1 Rubric (30 points)

Criterion	Points
Input handling — 3 prompts, no crash	4
Type conversion — quantity and price to `float`	3
Subtotal calculation (`qty × price`)	4
Service fee addition (+$2.50)	3
Tax calculation (8% on subtotal-with-fee)	4
Total calculation (subtotal-with-fee + tax)	3
Output formatting (`:.2f` , labels, readable layout)	3
Code quality (descriptive names, logical flow)	3
Passing score	**24 / 30**

Session 4 Wrap-Up

What We Covered Today

Hour 13: Lists

- List creation, indexing, slicing, and mutability
- `append()`, `remove()`, `pop()`, `in`, `len()`

Hour 14: Boolean Expressions

- All six comparison operators; `=` vs `==`
- `and`, `or`, `not` with truth tables and precedence

Hour 15: Conditionals

- `if / elif / else` flow; order-matters rule

Scope Guardrail Reminder

Stay in Basics Scope ✓

- Lists (no dictionaries yet)
- Boolean and comparison operators
- `if / elif / else` branching
- Arithmetic, type conversion, f-string formatting

Not Yet (Advanced Topics) ✗

- List comprehensions
- Lambda functions
- Decorators / generators
- Advanced OOP patterns
- Exception handling (`try / except`)

Next Session Preview

Session 5 (Hours 17–20)

- Hour 17: `for` loops — iterating over lists and ranges
- Hour 18: `while` loops and loop control (`break`, `continue`)
- Hour 19: Functions — defining, calling, parameters, return values
- Hour 20: Putting it together — small programs using loops + functions

Questions?

Key reminders from Session 4:

- Check list membership with `in` before calling `remove()`
- Single `=` assigns, double `==` compares
- Order `if / elif` from most specific to most general
- Always convert `input()` to the correct type before math
- Calculate at full precision — only round when printing

Thank You!

See you in Session 5!